



AWS SECURITY & PLATFORM ENGINEERING

Secure AWS Account Organization Guide

IAM, Roles, Automation, and DevSecOps Governance



A secure-by-default operating model for root-user isolation, workforce federation, workload roles, least-privilege authorization, CI/CD federation, and repeatable account governance as code.

DOCUMENT	AUDIENCE	Cloud, Security, Platform, and DevSecOps teams
TYPE	Implement-	
	ation guide	
EDITION	PDF edi-	
	tion - August	
	2026	

DOCUMENT NAVIGATION

Contents

1. Purpose and design principles

2. Recommended account structure

- 2.1 Minimum secure single-account layout
 - 2.2 Target multi-account layout
-

3. Root user controls

- 3.1 Root user policy
 - 3.2 Centralized root access for Organizations
 - 3.3 Root break-glass workflow
-

4. Human authentication and role model

- 4.1 Use IAM Identity Center
 - 4.2 Permission-set catalog
 - 4.3 Separation of duties
-

5. Workload and system authentication

- 5.1 Temporary credentials only
 - 5.2 One role per workload function
 - 5.3 Trust policy is a security boundary
-

6. IAM policy architecture

- 6.1 Policy layers
 - 6.2 Least privilege lifecycle
 - 6.3 Managed versus inline policies
 - 6.4 Permissions boundaries
 - 6.5 ABAC and tags
 - 6.6 Conditions
-

7. Operations role separation

- 7.1 Security role
 - 7.2 Operations role
 - 7.3 Deployment role
 - 7.4 CloudFormation execution role
-

8. Configuration repository and GitOps controls

- 8.1 Source-of-truth rules
 - 8.2 Pipeline stages
 - 8.3 Branch and environment model
-

9. DevSecOps policy tests

10. Auditing and continuous control

- Daily/continuous
 - Weekly
 - Monthly
 - Quarterly
-

11. IAM API and automation map

12. Implementation plan

- Phase 0 - Discovery
 - Phase 1 - Root and administrator safety
 - Phase 2 - Role migration
 - Phase 3 - Policy hardening
 - Phase 4 - Governance as code
 - Phase 5 - Multi-account governance
-

13. Acceptance criteria

1. Purpose and design principles

This guide describes how to organize an AWS environment so root-user activities, human administration, operations, workload access, and automation are separated. The design is suitable for a single AWS account but deliberately creates an upgrade path to AWS Organizations and a multi-account landing zone.

The governing principles are:

1. **Root is an emergency and root-only identity, not an administrator identity.**
2. **Human users authenticate centrally and receive temporary credentials by assuming roles.**
3. **Workloads receive temporary credentials from roles attached to their runtime.**
4. **Every permission is explicit, scoped, tagged, reviewed, and tested.**
5. **Guardrails limit maximum permissions; identity and resource policies grant the actual permissions.**
6. **Configuration is stored as code and deployed through a reviewed pipeline.**
7. **Logs, findings, inventories, and access activity feed continuous permission reduction.**
8. **Security is designed into account structure and delivery workflows rather than added later.**

The AWS IAM User Guide explains that authentication identifies the principal and authorization evaluates applicable policies. Requests are denied by default, explicit allows grant access, and an explicit deny overrides allows. Service control policies, permissions boundaries, and session policies can further reduce permissions. The guide's request-flow diagram illustrates how the principal, action, resource, policies, and request context are evaluated together.

2. Recommended account structure

2.1 Minimum secure single-account layout

For a temporary single-account environment, logically separate:

- **Root and recovery control**
- **Identity administration**
- **Security audit**
- **Platform administration**
- **Operations**
- **Application deployment**
- **Workload execution**
- **Billing and cost visibility**
- **Read-only support**

Use IAM Identity Center groups and permission sets for people. Use IAM roles for workloads and automation. Use tags, paths, naming conventions, permission boundaries, and separate CloudFormation stacks to maintain administrative boundaries inside the account.

Recommended IAM paths:

```
/org/security/
/org/platform/
/org/operations/
/org/workloads/<application>/
/org/automation/
/org/break-glass/
```

Recommended role names:

```
org-security-audit
org-platform-admin
org-operations
org-readonly
org-cfn-execution
org-github-deploy
app-<application>-<environment>-runtime
app-<application>-<environment>-deploy
```

2.2 Target multi-account layout

As the environment grows, move from logical separation to account boundaries:

```
Organization root
├── Security OU
│   ├── Log Archive account
│   └── Security Tooling / Audit account
├── Infrastructure OU
│   ├── Network account
│   └── Shared Services account
├── Workloads OU
│   ├── Development accounts
│   ├── Test/Staging accounts
│   └── Production accounts
├── Sandbox OU
└── Suspended / Quarantine OU
```

Use the Organizations management account only for organization-level functions, billing, IAM Identity Center administration, delegated administration, and account provisioning. Do not host production workloads there. Apply SCPs to OUs and member accounts, recognizing that SCPs do not apply to the management account and do not grant permissions.

For a prescriptive landing zone, AWS Control Tower can create the management, log archive, and audit/shared security accounts, and apply controls. CloudFormation StackSets can distribute common IAM, logging, configuration, and security resources to accounts and Regions.

3. Root user controls

3.1 Root user policy

The root user must be treated as a sealed recovery identity:

- Use a unique, strong password stored in an enterprise password vault.
- Register phishing-resistant MFA where supported; maintain more than one recovery method according to organizational policy.
- Do not create root access keys.
- Use a group-controlled email address, not a personal mailbox.
- Protect email, phone, DNS, domain registrar, and support/recovery channels with separate administrators and MFA.
- Restrict access to password-vault entries using multi-person approval.
- Alert on root sign-in and root API activity.
- Review root-related CloudTrail events after every authorized use.
- Use root only for AWS tasks that explicitly require root credentials.

3.2 Centralized root access for Organizations

For member accounts, enable centralized root access management. New member accounts created through Organizations can be created without root credentials. For existing member accounts, remove root credentials after prerequisites and recovery procedures have been tested.

Delegate root-access administration to a dedicated security account rather than operating it routinely from the management account. Privileged root actions should require a ticket, peer approval, a time-bounded session, and post-action evidence.

3.3 Root break-glass workflow

A root session must follow:

1. Incident or approved root-only task is documented.
2. Two authorized people approve access.
3. Vault credentials and MFA are released.
4. A screen recording or command transcript is retained where policy permits.
5. Only the approved root-only action is performed.
6. The session is terminated.

7. CloudTrail events are reviewed.
8. Password, recovery, and vault state are verified.
9. The ticket is closed with evidence.

Do not use the root user to recover from ordinary IAM mistakes that can be resolved through an administrator role or centralized privileged root session.

4. Human authentication and role model

4.1 Use IAM Identity Center

Use an organization instance of IAM Identity Center for workforce access. Connect an external identity provider when one exists; otherwise use the Identity Center directory.

Create groups, not per-user permission assignments:

```
AWS-Platform-Admins
AWS-Security-Auditors
AWS-Operations
AWS-Developers
AWS-ReadOnly
AWS-Billing-Viewers
AWS-BreakGlass-Approvers
```

Enforce MFA at the identity provider or IAM Identity Center. Automate joiner, mover, and leaver processes through the corporate identity lifecycle. Avoid IAM users for normal workforce access.

4.2 Permission-set catalog

Permission set	Intended use	Typical duration	Key restrictions
Administrator	Rare platform administration	1 hour	MFA, approval, no daily use
SecurityAudit	Security review and evidence	4 hours	Mostly read-only; no workload mutation
Operations	Routine operational changes	2-4 hours	No IAM/Organizations/root management
DeveloperPowerUser	Non-production application work	4-8 hours	No identity administration; environment scoped
ReadOnly	Support and investigation	8 hours	No mutation
BillingReadOnly	Cost visibility	4-8 hours	No payment/account changes

Permission set	Intended use	Typical duration	Key restrictions
EmergencyAdministrator	IdP outage only	shortest practical	Alerted, vaulted, post-use rotation

Use permission sets to create the IAM roles that people assume. Do not attach permissions directly to individuals except in a documented exception.

4.3 Separation of duties

At minimum, separate:

- Identity administrators from workload operators.
- Security reviewers from deployers.
- Production deployers from code authors through protected environments and approvals.
- Billing viewers from payment/account administrators.
- Root approvers from ordinary administrators.
- CI/CD deployment roles from CloudFormation execution roles.

No individual should be able to author, approve, and deploy a high-risk IAM change without an independent control.

5. Workload and system authentication

5.1 Temporary credentials only

Use the runtime-native role mechanism:

- EC2: instance profile.
- ECS: task role and separate task execution role.
- Lambda: execution role.
- EKS: EKS Pod Identity or IAM roles for service accounts.
- Step Functions: state-machine role.
- AWS services: service roles or service-linked roles.
- On-premises or non-AWS workloads: IAM Roles Anywhere where appropriate.
- GitHub Actions, GitLab, or compatible CI: OIDC federation.
- Cross-account workloads: assume a role in the target account.

Do not place access keys in source code, container images, Kubernetes Secrets, EC2 user data, Lambda environment variables, or CI/CD secret stores when role-based federation is available.

5.2 One role per workload function

Separate roles by application, environment, and function:

```
app-orders-dev-runtime
app-orders-prod-runtime
app-orders-prod-migrations
app-orders-prod-deploy
```

A runtime role should not deploy infrastructure. A deploy role should not read production business data unless the deployment requires it. Database migration roles should be activated only during controlled migration jobs.

5.3 Trust policy is a security boundary

A role has two distinct control planes:

- **Trust policy:** who can assume the role and under what conditions.
- **Permissions policies:** what the assumed role can do.

Restrict trust with exact account IDs, principal ARNs, service principals, OIDC audience, repository, branch, environment, source identity, external ID, source account, and source ARN as applicable.

For GitHub OIDC, restrict at least:

```
{
  "StringEquals": {
    "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
  },
  "StringLike": {
    "token.actions.githubusercontent.com:sub": "repo:ORG/REPO:ref:refs/heads/main"
  }
}
```

Use GitHub environments and environment-specific `sub` claims for production, with required reviewers.

6. IAM policy architecture

6.1 Policy layers

Use each policy type for its intended purpose:

1. **SCPs/RCPs:** organization-wide maximum-permission guardrails.
2. **Permissions boundaries:** maximum permissions for delegated IAM roles/users.
3. **Identity-based policies:** permissions granted to roles and groups.

4. **Resource-based policies:** resource-specific and cross-account access.
5. **Session policies:** further reduction for a particular session.
6. **Trust policies:** role assumption.
7. **KMS key policies and service-specific access controls:** service resource authorization.

The effective permission is the intersection of applicable allows, with explicit deny taking precedence.

6.2 Least privilege lifecycle

Do not attempt perfect least privilege on day one by guessing. Use an iterative process:

1. Start with a reviewed AWS managed policy or a controlled broader custom policy.
2. Record actual use through CloudTrail.
3. Generate a candidate policy with IAM Access Analyzer.
4. Review actions, resources, and conditions.
5. Validate the policy.
6. Simulate critical actions.
7. Deploy to a test environment.
8. Observe failures and unused access.
9. Narrow production permissions.
10. Repeat periodically.

6.3 Managed versus inline policies

Prefer customer-managed policies for reusable access patterns. Avoid inline policies except when a policy must have a strict one-to-one lifecycle with the identity. Version policies through Git; do not treat the AWS console as the source of truth.

6.4 Permissions boundaries

Attach a boundary to roles that developers or automated systems are allowed to create. A boundary does not grant access; it prevents delegated identities from exceeding the approved ceiling.

The starter boundary denies identity administration, Organizations/account administration, disabling major security controls, and destructive KMS actions. Adapt it to your organization and test it against workload requirements.

6.5 ABAC and tags

Use a hybrid RBAC + ABAC model:

- RBAC defines job/function roles.
- ABAC restricts access to resources whose tags match principal/session tags.

Recommended controlled tags:

```
Environment = dev | test | staging | prod
Application = orders
Owner = platform-team
CostCenter = CC-1234
DataClassification = public | internal | confidential | restricted
ManagedBy = CloudFormation
SecurityTier = 1 | 2 | 3
```

Require tags at resource creation where services support it. Prevent unauthorized modification of security-critical tags. Pass validated identity-provider attributes as session tags when using ABAC.

6.6 Conditions

Use conditions to reduce risk:

- `aws:MultiFactorAuthPresent` or identity-provider MFA context.
- `aws:PrincipalOrgID`.
- `aws:PrincipalArn`.
- `aws:SourceAccount` and `aws:SourceArn`.
- `aws:RequestedRegion`.
- `aws:ResourceTag/*`, `aws:RequestTag/*`, `aws:TagKeys`.
- `sts:ExternalId` for third parties.
- `sts:SourceIdentity`.
- OIDC claims for CI/CD.
- VPC endpoint conditions only after evaluating service behavior and break-glass paths.

7. Operations role separation

7.1 Security role

The security audit role may:

- Read IAM identities, policies, credential reports, Access Analyzer findings, CloudTrail, Config, Security Hub, GuardDuty, KMS metadata, and resource policies.
- Generate reports and open findings.
- Not alter production workload resources by default.

A separate security administrator role can manage security services but should not be a general workload administrator.

7.2 Operations role

The operations role may:

- Start, stop, restart, scale, and inspect approved workload resources.
- Read logs and metrics.
- Execute approved runbooks.
- Assume narrower incident-response roles.

It must not:

- Create users or access keys.
- Modify role trust policies.
- Attach `AdministratorAccess`.
- Disable CloudTrail, Config, GuardDuty, Security Hub, or Access Analyzer.
- Modify Organizations or root-access configuration.

7.3 Deployment role

The CI/CD deployment role should:

- Be assumed through OIDC.
- Be restricted to one repository and branch/environment.
- Create a CloudFormation change set.
- Pass only an approved CloudFormation execution role.
- Deploy only stack-name prefixes and target accounts/Regions approved for that pipeline.
- Use a separate role per environment and account.
- Avoid direct IAM mutations when CloudFormation can perform them through the execution role.

7.4 CloudFormation execution role

The execution role is assumed by CloudFormation, not by developers or GitHub directly. Scope it to the resource types, names, paths, tags, and accounts that its stacks manage.

This creates a two-role model:

```
GitHub OIDC principal
-> assumes deployment role
    -> creates/executes change set
        -> CloudFormation assumes execution role
            -> changes approved resources
```

The deployment role can pass only the exact execution role. The execution role cannot be assumed by GitHub.

8. Configuration repository and GitOps controls

8.1 Source-of-truth rules

- All IAM policies, trust policies, permission sets, boundaries, SCPs, and role definitions are stored in Git.
- Console changes are break-glass exceptions and must be reconciled back to code.
- Production branches are protected.
- IAM/security paths require CODEOWNERS approval.
- Pull requests include risk, affected principals, actions, resources, rollback, and validation evidence.
- Secrets are never committed.
- Changes are deployed from immutable commits.
- CI actions and dependencies should be pinned to immutable versions or commit SHAs.
- Artifact integrity and provenance should be retained.

8.2 Pipeline stages

1. Format and parse JSON/YAML.
2. Secret scanning.
3. CloudFormation linting.
4. IAM Access Analyzer validation.
5. Static policy rules, such as blocking wildcard admin grants.
6. CloudFormation change-set generation.
7. Human review of IAM capabilities and resource replacements.
8. Deployment to sandbox.
9. Automated smoke and authorization tests.
10. Promotion to production with protected-environment approval.
11. Post-deployment inventory and drift checks.
12. Evidence retention.

8.3 Branch and environment model

```
feature/* -> pull request validation
main      -> deploy sandbox/shared-dev
release   -> deploy staging after approval
tag       -> deploy production after security/operations approval
```

Prefer separate repositories or directories for:

- Organization guardrails.
- Account baseline.
- Identity Center configuration.
- Application infrastructure.
- Workload policies.

The organization guardrail repository should have the strictest ownership.

9. DevSecOps policy tests

Every policy change should answer:

- Does it use `Action: "*" ?`
- Does it use `Resource: "*" ?` for actions that support resource-level permissions?
- Can it create or update IAM roles, policies, access keys, identity providers, or permission sets?
- Can it call `iam:PassRole` on more roles than required?
- Can it change a role trust policy?
- Can it disable logging or security services?
- Can it decrypt broadly with KMS?
- Can it access production from a non-production principal?
- Does a trust policy accept an entire external account or repository unnecessarily?
- Are OIDC audience and subject claims restricted?
- Does the change create new public or cross-account access?
- Is there an explicit rollback?

Use IAM Access Analyzer validation and custom checks, policy simulation, CloudFormation change sets, and test-role sessions to verify the answer.

10. Auditing and continuous control

| Daily/continuous

- Alert on root activity, access-key creation, IAM policy changes, trust-policy changes, OIDC/SAML provider changes, and security-service disablement.
- Review Access Analyzer external-access findings.
- Detect CI/CD role assumption from unexpected subjects, branches, repositories, Regions, or source IP ranges where meaningful.
- Monitor failed authorization spikes.

Weekly

- Review newly created roles and policies.
- Review roles with wildcard actions/resources.
- Review CloudFormation drift and out-of-band IAM changes.
- Review failed and denied deployment events.

Monthly

- Generate and review the IAM credential report.
- Find unused passwords and access keys.
- Review role last-used data.
- Remove stale roles and policies.
- Review external and cross-account trust.
- Review emergency and administrator role usage.
- Confirm no root access keys exist.

Quarterly

- Re-certify Identity Center group membership and assignments.
- Run access reviews for production and security roles.
- Generate least-privilege candidates from access activity.
- Review SCP and permission-boundary effectiveness.
- Test break-glass and root-access procedures.
- Review account recovery contacts and vault access.
- Validate delegated administrator configuration.

11. IAM API and automation map

The IAM API reference provides the building blocks for automation. Common operations include:

- Create and manage roles: `CreateRole`, `UpdateAssumeRolePolicy`, `AttachRolePolicy`, `PutRolePolicy`, `TagRole`.
- Create policies and versions: `CreatePolicy`, `CreatePolicyVersion`, `SetDefaultPolicyVersion`.
- Apply maximum-permission controls: `PutRolePermissionsBoundary`.
- Create federation: `CreateOpenIDConnectProvider`, `CreateSAMLProvider`.
- Validate behavior: `SimulatePrincipalPolicy`, `SimulateCustomPolicy`.
- Audit credentials: `GenerateCredentialReport`, `GetCredentialReport`, `ListAccessKeys`, `GetAccessKeyLastUsed`.

- Review access activity: `GenerateServiceLastAccessedDetails`, `GetServiceLastAccessedDetails`.
- Inventory: `GetAccountAuthorizationDetails`, `ListRoles`, `ListUsers`, `ListPolicies`.

Use AWS SDKs or AWS CLI rather than hand-building signed Query API requests. Treat IAM changes as setup/control-plane operations and allow for IAM eventual consistency before production workflows depend on newly created identities or policies.

12. Implementation plan

Phase 0 - Discovery

- Inventory root status, IAM users, access keys, roles, trust relationships, policies, and identity providers.
- Identify applications using static keys.
- Export account authorization details and credential reports.
- Identify business owners and recovery owners.
- Classify environments and critical resources.

Phase 1 - Root and administrator safety

- Secure root password, MFA, mailbox, phone, and support access.
- Remove root access keys.
- Create emergency runbook and approvals.
- Enable IAM Identity Center.
- Create administrator and audit groups.
- Stop normal root and IAM-user administration.

Phase 2 - Role migration

- Replace human IAM users with Identity Center access.
- Replace EC2/ECS/Lambda/EKS static keys with runtime roles.
- Replace CI access keys with OIDC.
- Use IAM Roles Anywhere for eligible external workloads.
- Deactivate and then delete obsolete keys after validation.

Phase 3 - Policy hardening

- Establish role catalog and naming/path/tag standards.
- Create workload permission boundary.

- Refactor reusable customer-managed policies.
- Add conditions and resource scoping.
- Enable IAM Access Analyzer.
- Generate policies from CloudTrail activity.

Phase 4 - Governance as code

- Import or recreate configuration in CloudFormation.
- Add validation, CODEOWNERS, protected branches, and change sets.
- Add CI/CD deployment and CloudFormation execution roles.
- Add drift detection and reconciliation.

Phase 5 - Multi-account governance

- Create Organizations OUs and dedicated accounts.
- Enable centralized root access for member accounts.
- Delegate security administration.
- Apply tested SCPs.
- Use StackSets or Control Tower customization to replicate baselines.
- Move workloads out of the management account.

13. Acceptance criteria

The account is considered organized when:

- Root is not used for daily administration and has no access keys.
- All human access is federated, MFA-protected, and group-based.
- All workloads use temporary role credentials unless a documented exception exists.
- CI/CD uses OIDC and a two-role deployment/execution model.
- IAM resources are tagged, named consistently, and defined in Git.
- Production IAM changes require independent approval.
- Permission policies pass automated validation.
- Access Analyzer is enabled and findings have owners.
- Credential and unused-access reviews run on a schedule.
- Break-glass and rollback procedures have been tested.
- No production workloads run in the Organizations management account.